🚗 UBER REAL-TIME DATA ENGINEERING PROJECT

☁️ End-to-End Data Engineering with Azure & Databricks

My first end-to-end data engineering project—bringing together batch ingestion, ride-event streaming, Spark transformations, and star-schema modeling in an automated workflow.

🏗️ Project Architecture

Uber Data Engineering Project Architecture

📖 About the Project

I built this project to get hands-on experience with how the different parts of a data engineering platform work together—from collecting data to preparing it for analytics.

The project combines batch data from GitHub with ride events sent through Azure Event Hubs. Azure Data Factory handles batch ingestion, Azure Blob Storage stores the ingested files, and Databricks provides the Spark processing environment. The final output is organized into a star schema.

🛠️ Tools & Technologies

Technology	How I Used It
🐍 Python	Sending ride events and writing supporting code
⚡ Azure Event Hubs	Ingesting ride events
🏭 Azure Data Factory	Batch ingestion and workflow orchestration
🗄️ Azure Blob Storage	Storing ingested JSON files
🧱 Azure Databricks	Developing and running data processing workloads
🔥 PySpark	Selecting, transforming, and deduplicating data
🌊 Spark Structured Streaming	Processing streaming data
📐 Spark Declarative Pipelines	Building declarative processing pipelines
🐙 GitHub	Source data and project code

🔄 Data Flow

📦 Batch Ingestion

Source data from GitHub is ingested using Azure Data Factory and stored in Azure Blob Storage before Spark processing.

```
GitHub → Azure Data Factory → Azure Blob Storage → Databricks
```

⚡ Event Ingestion

A Python producer sends ride events to Azure Event Hubs for processing with Spark Structured Streaming.

```
Python Ride Producer → Azure Event Hubs → Spark Structured Streaming
```

✨ Transformation & Modeling

Data is processed using PySpark and Spark Declarative Pipelines. The workflow prepares ride-related data and organizes the final output into a star schema for analytics.

✅ What I Implemented

- 📦 Batch ingestion with Azure Data Factory.
- ⚡ Ride-event publishing to Azure Event Hubs.
- 🗄️ JSON data storage in Azure Blob Storage.
- 🔥 Spark processing in Databricks.
- 🌊 Streaming processing with Spark Structured Streaming.
- 📐 Declarative processing with Spark Declarative Pipelines.
- 🚗 Driver attribute selection and deduplication using `driver_id`.
- ⭐ Star-schema output.
- 🤖 Automatic pipeline execution.

🚧 Challenges I Worked Through

- Missing Event Hubs connection settings.
- Incorrect source URLs and HTTP 404 errors.
- SAS permissions and blob access errors.
- Dynamic file paths and Python string formatting.
- Undefined variables in notebook execution.
- Duplicate records during transformations.

Working through these issues helped me understand the connections between configuration, ingestion, storage access, and processing—not just the individual tools.

What I Learned

- How batch and streaming ingestion fit into one data platform.
- How Azure Data Factory, Event Hubs, storage, and Databricks connect.
- How to use PySpark for data transformations and deduplication.
- How to prepare analytical data using dimensional modeling.
- How to move from manually running notebooks to an automated pipeline.
- Why credential handling and careful debugging matter in cloud projects.

About This Project

This is a student learning and portfolio project, and my first completed end-to-end data engineering project. It is not affiliated with or endorsed by Uber.